1. Dockerfile

Vayan al repo de **Simulation** dentro del org de LiDRON y hagan **Save As** del file que se llama exactamente:

Hay tres Dockerfiles, bajen solo el que dice exactamente Dockerfile.

Para guía, sigan en el repo de Landing el docker_setup.md:

- Step 5: Obtain the Dockerfile in GitHub
- Step 6: Move the Dockerfile into the Project Directory

No hagan el Step 7.

2. **Build del container**

Entren al folder donde guardaron ese Dockerfile en Ubuntu.

Ejemplo:

```
cd gazebo-harmonic
docker build -t px4-gazebo .
```

3. Correr el container con el repo mounted.

```
docker run -it --name landing_dev \
  --net=host \
  -e DISPLAY=$DISPLAY \
  -e QT_X11_NO_MITSHM=1 \
  -v /tmp/.X11-unix:/tmp/.X11-unix \
  -v /mnt/c/Users/brian/OneDrive/Desktop/LiDron/Landing_repo/Landing:/Landing \
  px4-gazebo bash
```

Importante: no hagan copy-paste exacto del path.
Tienen que cambiar esta parte:

-v /mnt/c/Users/brian/OneDrive/Desktop/LiDron/Landing_repo/Landing:/Landing

por el path donde ustedes tengan su repo de Landing. Si entran al repo en vsc les sale el directory.

4. Entrar al repo de PX4 dentro del container

cd /PX4-Autopilot

5. Attachear el LiDAR al drone

Instalen nano si no lo tienen:

apt update && apt install -y nano

Luego habran:

nano /PX4-Autopilot/Tools/simulation/gz/models/x500_base/model.sdf

Copien el script del LiDAR ubicado en notepad y al final antes de </model> le hacen paste al script.

6. Empezar simulacion:

Terminal 1:

cd /PX4-Autopilot
make px4_sitl gz_x500

7. Bridge del pointcloud:

Terminal 2:

source /opt/ros/jazzy/setup.bash

ros2 run ros_gz_bridge parameter_bridge
/lidar/points/points@sensor_msgs/msg/PointCloud2@gz.msgs.PointCloudPacked

Dejen este terminal corriendo.
No hagan CNTRL + C.

8. Verificar que el point cloud está llegando

Terminal 3:
ros2 topic list

ros2 topic echo /lidar/points/points –once

Si sale un mensaje PointCloud2, entonces todo está funcionando bien.

9. Para correr y testiar

Para correr el LiDAR y extraer la data de Gazebo y usarla en los scripts de filter tiene que hacer los pasos 6 al 7.

Terminal 4:

colcon build

source install/setup.bash

ros2 run lidar_preprocessing lidar_preprocessing_node

Esto va imprimir unos logs (prints) de que esta funcionando.

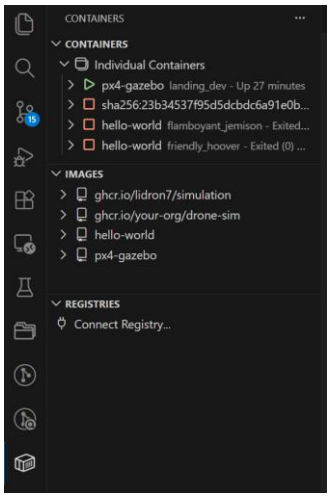
10. Trabajar en el package desde VS Code

Los cambios del package se hacen en el repo Landing/src/...

Si editan desde VS Code normal, puede que salgan errores en imports como rclpy porque Windows no tiene las dependencias de ROS.

Para eso:

- instalen la extensión de Docker en VS Code
- vayan al icono de Docker
- busquen el container corriendo
- right click
- Attach Visual Studio Code



Eso abre una nueva ventana de VS Code dentro del container.

Ahí:

- no salen errores de imports
- pueden correr y testear el código
- los cambios se reflejan en el repo original porque está mounted

11. Nota importante

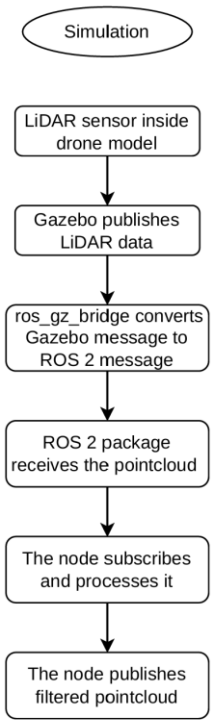
Si el container no está corriendo, primero hagan:

docker start landing_dev

y luego hacen attach desde VS Code.

How it works:

Gazebo Simulation:



What is ROS2 Package?

A ROS2 package is basically a project folder that ROS understands.

The package is were your code lives. The main purpose of this package is to create nodes.

Think of it like this:

Normal python project – Python understands it

ROS2 package – ROS understands it too

Package folder structure:

```
Landing/
└── src/
    └── lidar_preprocessing/
        ├── package.xml
        ├── setup.py
        ├── resource/
        └── lidar_preprocessing/
            ├── __init__.py
            └── lidar_preprocessing_node.py
```

src/

Standard ROS2 workspace source folder.

lidar_preprocessing folder

This folder is the package root. The actual python module folder.

package.xml

This is the package description file. ROS uses this file to understand what your package needs.

It tells ROS things like:

- package name
- version
- dependencies
- maintainer
- description

setup.py

This tells Python and ROS how to install the package.

That means when you run:
ros2 run lidar_preprocessing lidar_preprocessing_node

ROS goes to package lidar_preprocessing the file lidar_preprocessing_node and runs the main() function.

__init__.py

Tells python that the folder is a Python package/module. It can be empty, that is normal.

lidar_preprocessing_node.py

This the main code file, where the ROS node lives.

What the LiDAR script in the model does:

This script tells gazebo:

- add a LiDAR sensor to the drone
- place it at a certain position on the drone
- define scan settings
- define range settings
- define update rate
- publish scan data

The Topic:

This tells Gazebo what topic to publish on. This is why we use /lidar/points/points.

What the bridge does:

Gazebo and ROS 2 do not speak the same message system directly.

Gazebo uses Gazebo transport messages like:

- gz.msgs.PointCloudPacked
- gz.msgs.LaserScan

ROS 2 uses ROS message types like:

- sensor_msgs/msg/PointCloud2

So you need a translator. The translator is the bridge.

You ran:

ros2 run ros_gz_bridge parameter_bridge /lidar/points/points@sensor_msgs/msg/PointCloud2@gz.msgs.PointCloudPacked

This means:

Take the Gazebo topic /lidar/points/points, which is gz.msgs. PointCloudPacked, and expose it to ROS 2 as sensor_msgs/msg/PointCloud2.

This is why the node is able to read the LiDAR information.